Laboratorio Spring Boot con Cursor
2026
Tecnológico de Antioquia

Tecnologías Utilizadas

Backend

- **Java 25+**
- **Spring Boot 4**
- **Spring Data JPA**
- **PostgreSQL** (Base de datos)

Frontend

- **NodeJS 24+**
- **Angular 21+**
- **TypeScript**

Guía de Instalación de Herramientas

Esta guía detalla los pasos necesarios para configurar tu entorno de desarrollo para el curso de Aplicaciones FullStack.

1. Java Development Kit (JDK) 25

El JDK proporciona las herramientas necesarias para compilar y ejecutar aplicaciones Java. En este curso lo usamos para desarrollar y poner en marcha el backend basado en Spring Boot; sin él no se podría construir ni lanzar la API.

- **Descarga**: Accede a la web oficial de Oracle o utiliza Adoptium (Temurin). Esta es la distribución del kit que incluye el compilador javac, la máquina virtual JVM y otras utilidades.
- **Instalación**: Ejecuta el instalador y marca la opción "Add to PATH" para que los comandos Java sean accesibles desde cualquier terminal. Esto añade las rutas necesarias a las variables de entorno.
- **Verificación**: Abre una terminal y ejecuta:

java -version

Con este comando confirmas que el ejecutable se encuentra en el PATH y ves la versión instalada, lo cual es útil para asegurarte de que tienes la versión requerida.

2. Node.js 24

Node.js es un entorno de ejecución de JavaScript en el servidor que incluye npm, el gestor de paquetes. Es fundamental para el frontend porque nos permite instalar librerías de Angular, compilar el proyecto y lanzar el servidor de desarrollo con ng serve.

- **Descarga**: Ve a nodejs.org y selecciona la versión 24 (elige la LTS para máxima estabilidad). Esta instalación incluye el runtime de Node y el administrador de paquetes npm.
- **Verificación**:
- node -v

npm -v

Estos comandos te muestran las versiones instaladas; confirma que ambas herramientas están disponibles antes de avanzar.

3. Git y GitHub

Git es el sistema de control de versiones que usamos para llevar un historial de los cambios en el código. GitHub es una plataforma remota que nos permite alojar repositorios, colaborar con otros y hacer copias de seguridad de nuestro trabajo.

- **Git**: Descárgalo desde git-scm.com. La instalación te dará acceso a comandos como git commit, git push y git pull.
- **GitHub**: Crea una cuenta en github.com si aún no la tienes; allí alojarás tus repositorios y podrás compartir proyectos y colaborar.

- **Configuración inicial**: Una vez instalado Git, define tu identidad global con estos comandos para que tus commits estén correctamente atribuidos:
- git config --global user.name "Tu Nombre"

git config --global user.email "tu@email.com"
También puedes configurar otros ajustes como el editor predeterminado o el formato de línea de finalización dependiendo de tu entorno.

4. Postgresql
PostgreSQL es un sistema de gestión de bases de datos relacionales de código abierto y potente. En este curso, lo utilizaremos para almacenar de forma persistente toda la información de nuestra aplicación (reservas, usuarios, productos, etc.), aprovechando su robustez y compatibilidad con Spring Data JPA.

- **Descarga**: Ve a la página oficial de PostgreSQL y selecciona el instalador para tu sistema operativo (Windows, macOS o Linux).
- **Instalación**:
    - Ejecuta el instalador y sigue los pasos del asistente.
    - **Contraseña del superusuario**: Durante la instalación, se te pedirá una contraseña para el usuario postgres. Asegúrate de recordarla, ya que la necesitarás para configurar la conexión en Spring Boot.
    - **Puerto**: Por defecto utiliza el 5432. Déjalo así a menos que ya esté ocupado.
    - **Stack Builder**: Al finalizar, no es necesario instalar complementos adicionales a través de Stack Builder para este curso.
- **Herramienta de Administración (pgAdmin)**: El instalador suele incluir **pgAdmin**, una interfaz gráfica que te permitirá crear la base de datos mi_basededatos de forma visual antes de conectar la aplicación.
- **Verificación**: Abre pgAdmin o utiliza la terminal (psql -U postgres) para confirmar que puedes acceder al servidor de base de datos.

5. Cursor
Cursor es un editor de código basado en VS Code que integra inteligencia artificial de forma nativa. Lo utilizaremos como nuestra herramienta principal de desarrollo para acelerar la escritura de código, realizar refactorizaciones inteligentes y resolver errores de manera eficiente tanto en el Backend como en el Frontend.

- **Descarga**: Ve a cursor.com y descarga la versión correspondiente a tu sistema operativo.
- **Instalación**: Ejecuta el instalador. Al ser un fork de VS Code, puedes importar todas tus extensiones, configuraciones y atajos de teclado existentes durante el primer inicio.
- **Configuración de IA**: Al iniciar, Cursor te permitirá configurar el modelo de lenguaje (como Claude 3.5 Sonnet o GPT-4o). Asegúrate de loguearte para aprovechar las funciones de autocompletado predictivo (Tab) y el chat contextual (Ctrl+L o Cmd+L).
- **Uso de Composer**: Familiarízate con la función Composer (Ctrl+I o Cmd+I), que permite generar cambios en múltiples archivos simultáneamente siguiendo instrucciones en lenguaje natural.

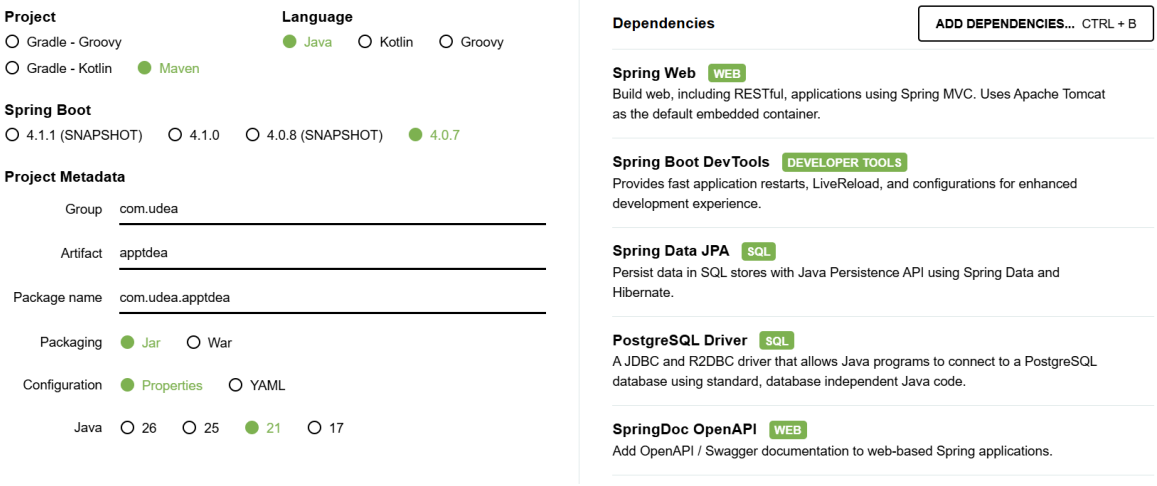
6. Extensiones necesarias
Para sacar el máximo partido a este curso, instala las siguientes extensiones en VS Code:

1. **Java Extension Pack** (incluye herramientas de lenguaje Java, Maven y Spring Boot). Permite autocompletar, depurar y ejecutar aplicaciones Spring.
2. **Spring Boot Extension Pack** o al menos **Spring Boot Tools**: facilita la creación y ejecución de proyectos Spring Boot.
3. **Angular Language Service**: proporciona autocompletado, navegación y comprobaciones en tus archivos .ts, .html y .css de Angular.

Laboratorio Spring Boot con Cursor
2026
Tecnológico de Antioquia

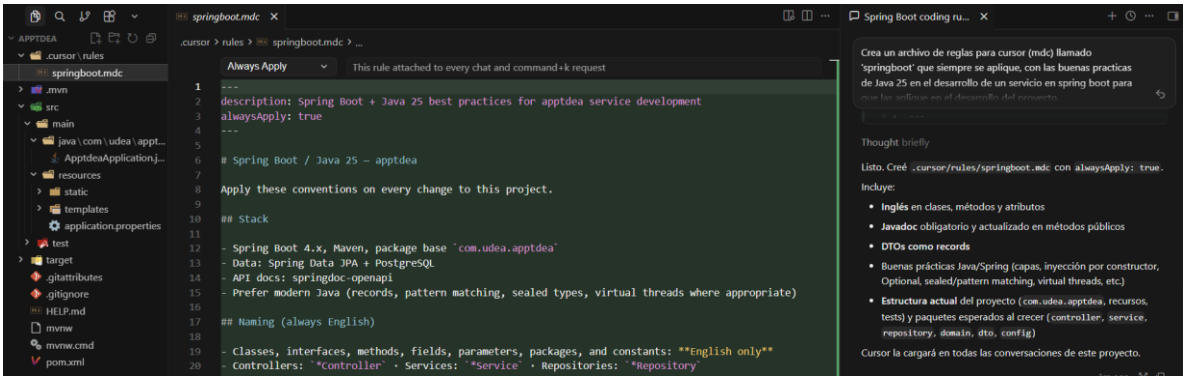
Puedes instalar todas estas extensiones a la vez desde el marketplace de VS Code o mediante la paleta de comandos (Ctrl+Shift+P) con Extensiones: Instalar extensiones y buscando por nombre.

Plantilla del proyecto



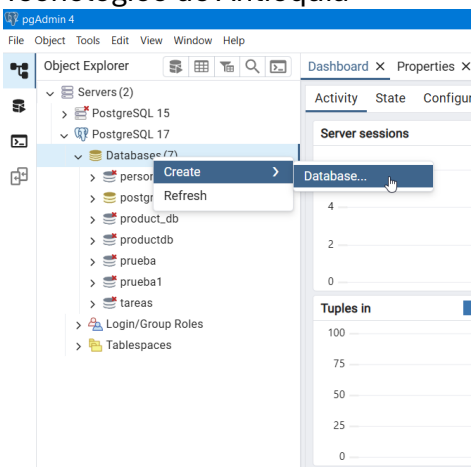
En cursor colocar :

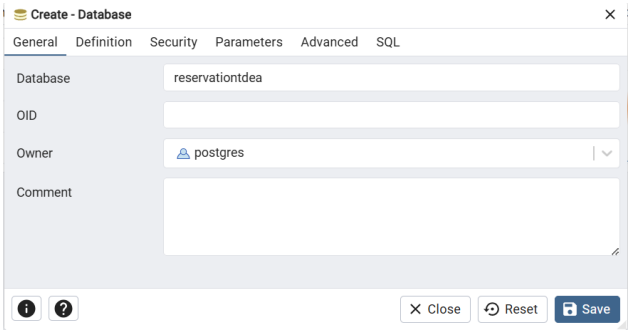
Crea un archivo de reglas para cursor (mdc) llamado 'springboot' que siempre se aplique, con las buenas practicas de Java 25 en el desarrollo de un servicio en spring boot para que las aplique en el desarrollo del proyecto.
Ten en cuenta que los nombres de clases, métodos y atributos deben ser en ingles.
Cada método público debe tener su documentación Javadoc actualizada.
Para los DTO usamos java records.
Agrega la estructura actual del proyecto dentro de las reglas.



Configurar la Base de Datos de PostgreSQL o la que corresponda a su gusto.

Laboratorio Spring Boot con Cursor
2026
Tecnológico de Antioquia





Prompt para generación del archivo **application.properties.**

Crea la configuración para la conexión a la base de datos postgresql de nombre 'reservationtdea' con el usuario 'postgres' y contraseña 'root'. Habilita la actualización del esquema automático en hibernate, el dialecto JPA e imprime las sentencias sql de JPA en consola.

Resultado generado

```
spring.application.name=apptdea

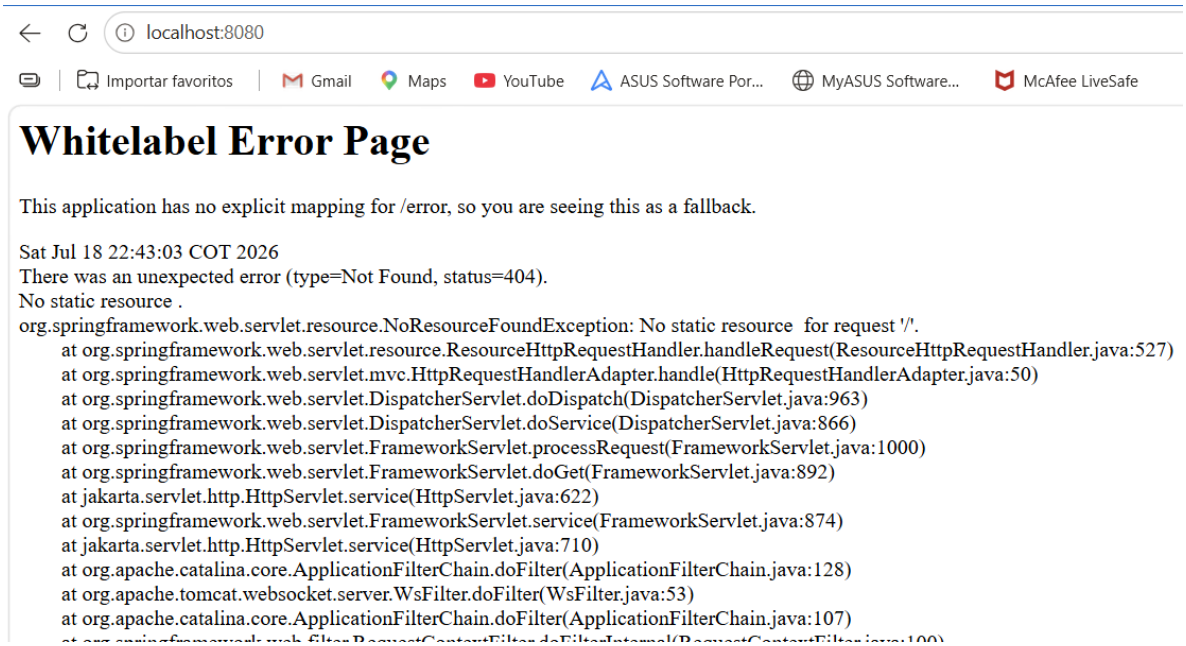
# PostgreSQL datasource
spring.datasource.url=jdbc:postgresql://localhost:5433/reservationtdea
spring.datasource.username=postgres
spring.datasource.password=root
spring.datasource.driver-class-name=org.postgresql.Driver

# JPA / Hibernate
spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
spring.jpa.open-in-view=false
```

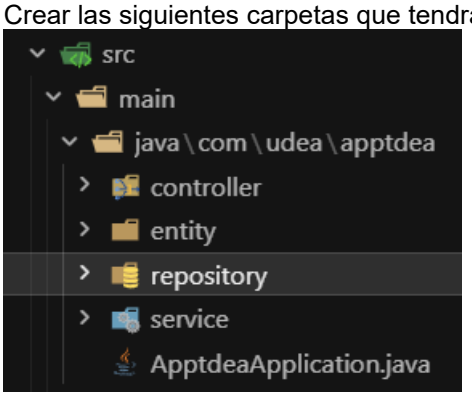
Laboratorio Spring Boot con Cursor
2026
Tecnológico de Antioquia

Ejecuta la APP y vea la salida por la consola.

Si ejecuta localhost:8080 aparece error porque no hay ninguna vista para cargar.



Crear las siguientes carpetas que tendrán la configuración de base del proyecto



Prompt para generar los Entity Beans

```
Crea una entidad JPA que represente una reserva del sistema.

Debe tener los siguientes atributos (ajusta los nombres):
- id: Long, clave primaria autogenerada
- customerName: String
- date: LocalDate
- time: LocalTime
- service: String
- ReservationStatus: un enum con valores ACTIVA y CANCELADA

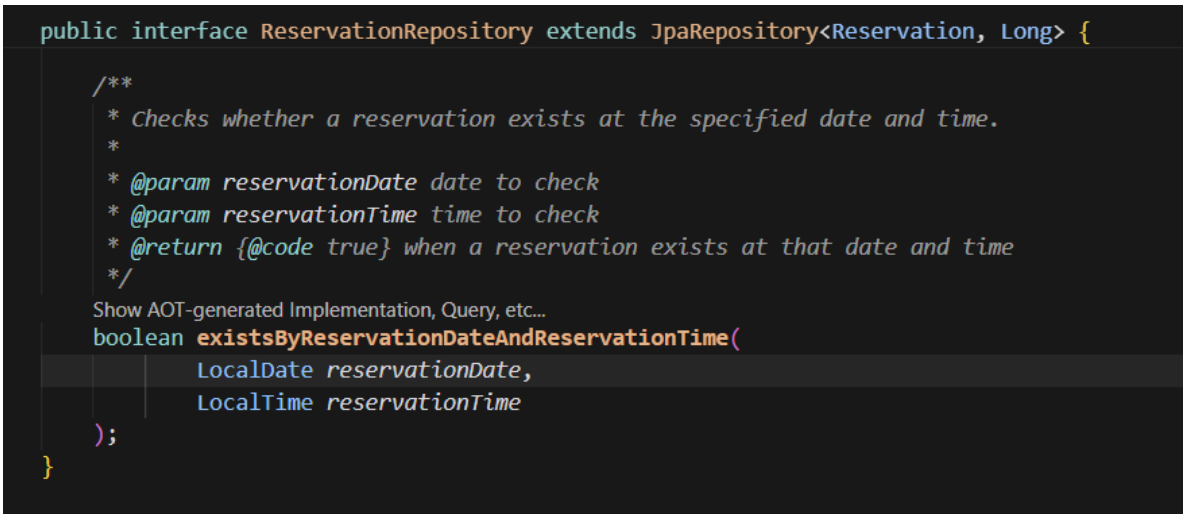
Generar getters, setters y constructores.
Usa las anotaciones JPA correctas.
```

Laboratorio Spring Boot con Cursor
2026
Tecnológico de Antioquia

Prompt para generar el Repository JPA

Crea un repositorio Spring Data JPA para la entidad Reserva.

Debe extender JpaRepository y tener un método que permita verificar si ya existe una reserva para una fecha y una hora específicas.



Prompt para generar la Capa Service
Crea una clase ReservaService que contenga la lógica de negocio del sistema.

Debe:
- Crear una reserva solo si no existe otra reserva en la misma fecha y hora
- Cancelar una reserva por su id
- Lanzar una excepción personalizada cuando se violen las reglas de negocio

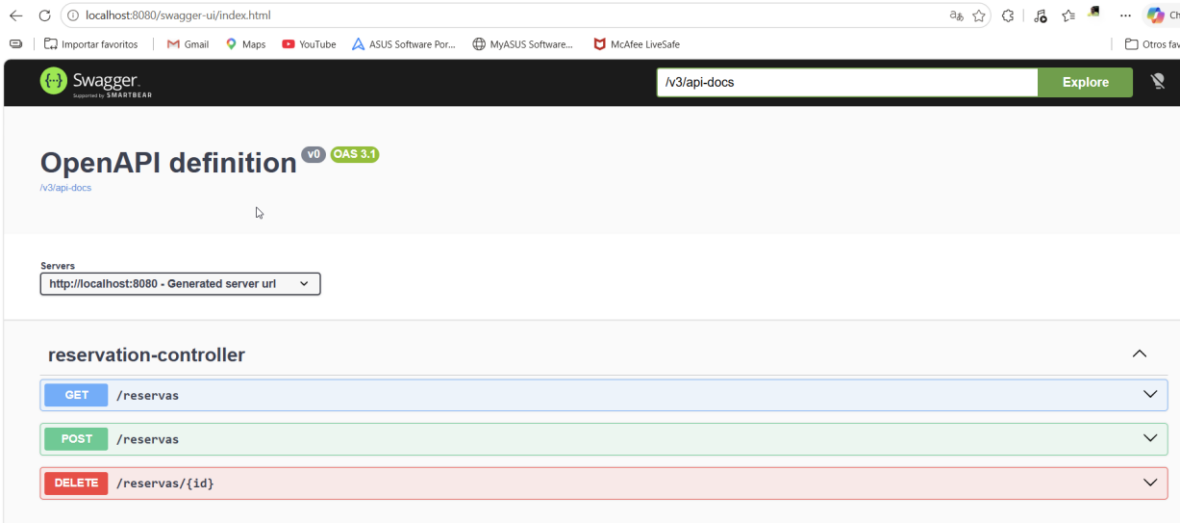
Prompt para generar la CAPA CONTROLLER
Crea un controlador REST para la entidad Reserva.

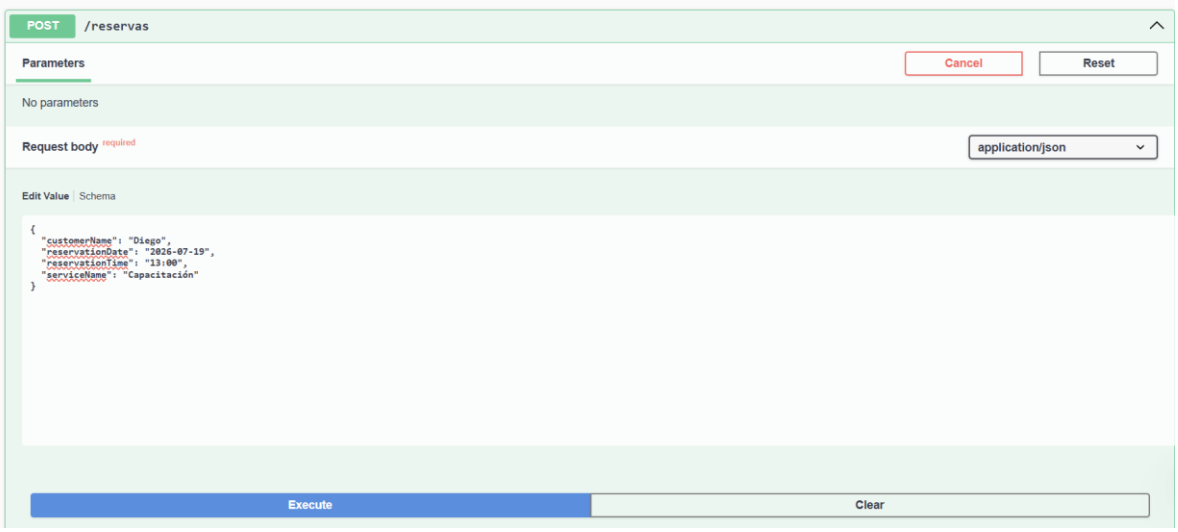
Debe exponer los siguientes endpoints:
- GET /reservas → listar todas las reservas
- POST /reservas → crear una nueva reserva
- DELETE /reservas/{id} → cancelar una reserva

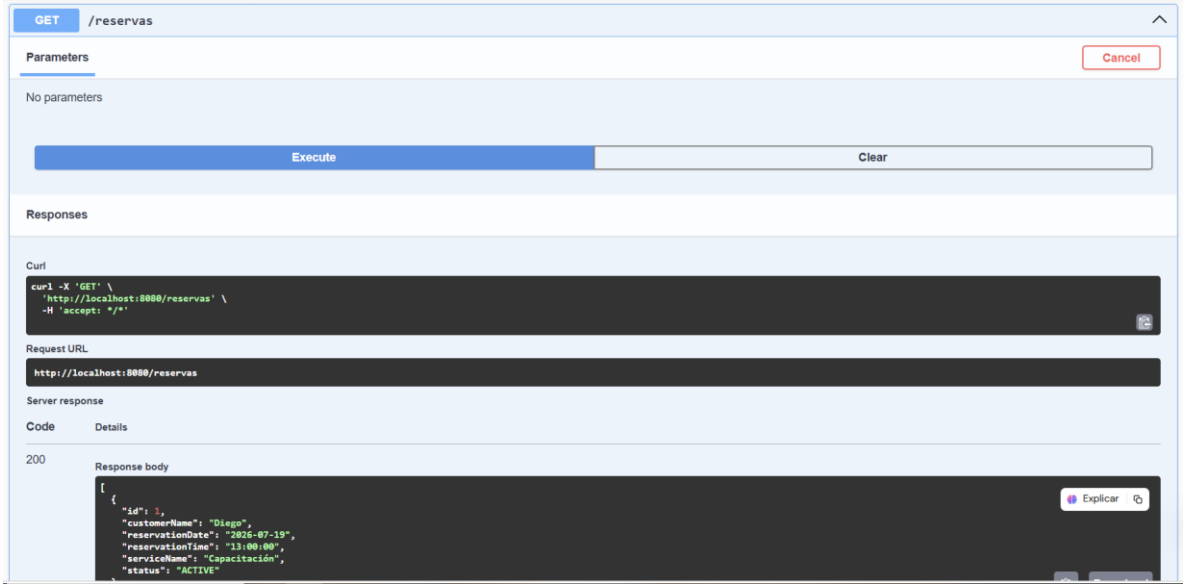
Devuelve códigos de estado HTTP adecuados en cada caso.

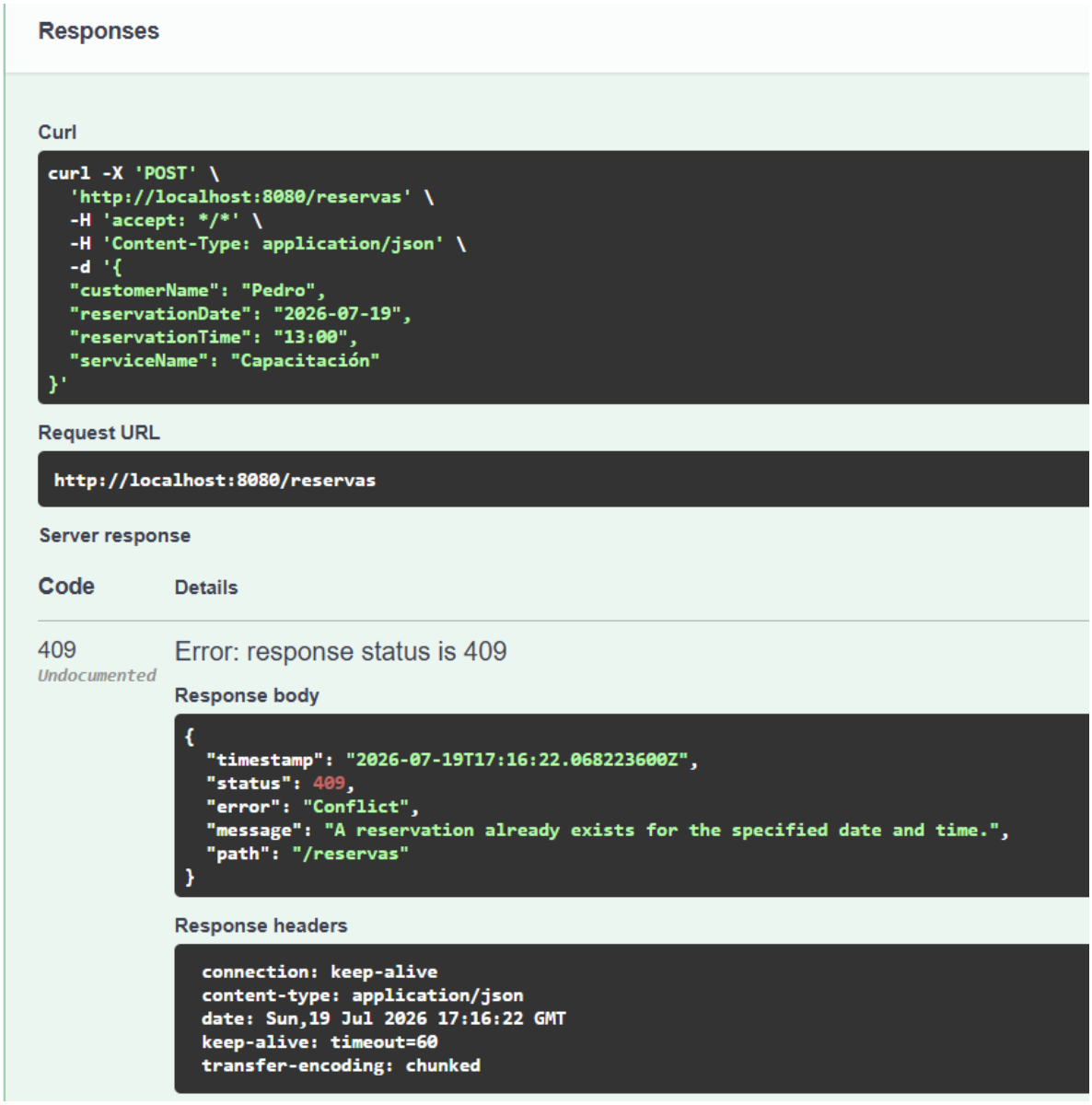
Ahora ejecuta la aplicación con swagger y prueba cada endpoint generado.

Laboratorio Spring Boot con Cursor
2026
Tecnológico de Antioquia



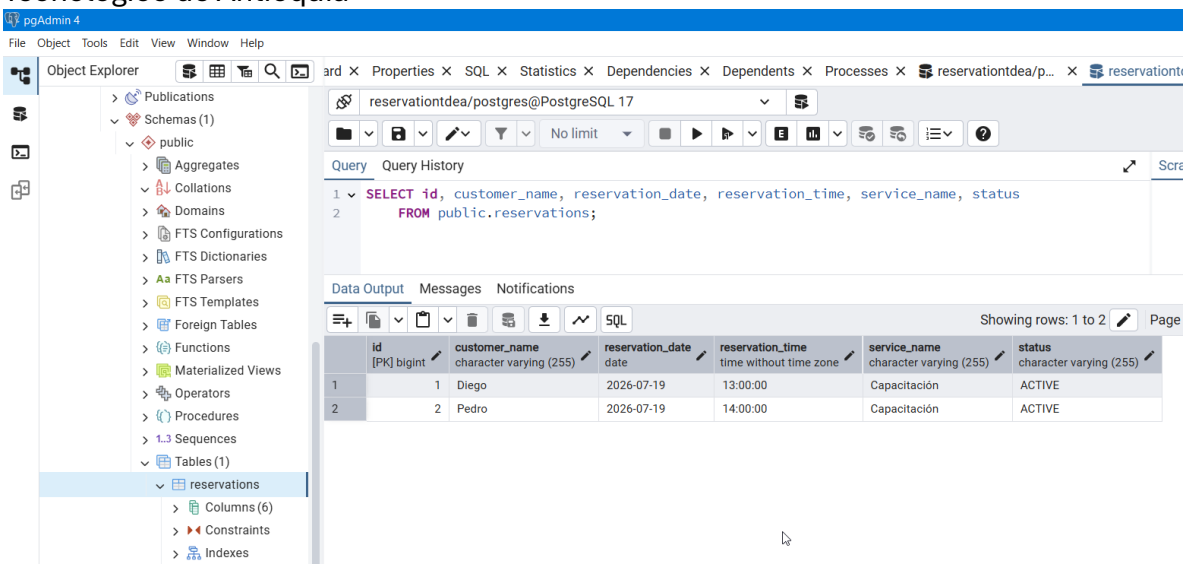






Revisa que todos los datos queden guardados en su respectiva base de datos:

Laboratorio Spring Boot con Cursor
2026
Tecnológico de Antioquia



Creación del Frontend con REACJS
Para generar el frontend debe crear el filtro CORS para evitar problemas en las llamadas hacia las rutas del Backend.

Prompt para crear el Filtro CORS
*Crea una clase Configuration para crear un filtro de CORS que se aplicará a todas las rutas (/**). Le indica a Spring Boot que debe aceptar peticiones del origen http://localhost:3000 y permite todos los métodos y cabeceras necesarios para la comunicación.*

Prompt para crear la estructura del proyecto con ReactJS

Usando @.cursor/rules/springboot.mdc como regla, crea un servicio en ReactJS llamado ReservaService.

Debe:
- Usar AXIOS
- Tener métodos para:
- Obtener todas las reservas
- Crear una reserva
- Cancelar una reserva
- Usar una variable de entorno para la URL del backend

Realiza las configuraciones adecuadas para que todo quede en funcionamiento.

Promtp para generar las vistas de React (Tabla de datos)

Crea una página en ReactJS que muestre una tabla con las reservas.

El componente debe:
- Obtener las reservas desde el ReservaService
- Mostrar los datos en una tabla dinamica
- Permitir cancelar una reserva mediante un botón
- Usar el servicio ReservaService

Laboratorio Spring Boot con Cursor
2026
Tecnológico de Antioquia

Recuerda que debemos tener separados los 3 archivos del componente (`.ts`, `.html` y `.css`)

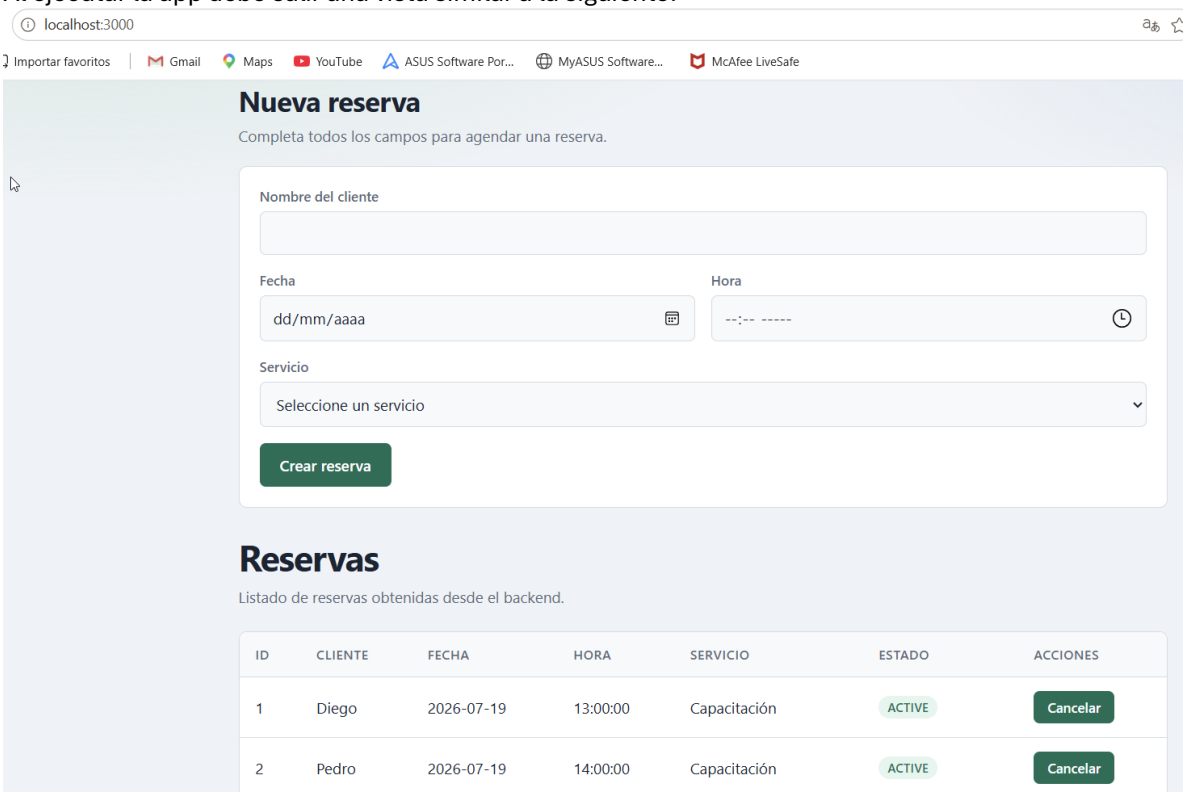
Promtp para generar las vistas de React (Formulario de creación de reservas)

Crea un formulario reactivo en ReactJS para crear una reserva.

Debe incluir los campos:
- nombreCliente
- fecha
- hora
- servicio: con una lista de selección con los posibles servicios que ofrecemos en nuestro sistema (recomienda algunos).

Todos los campos son obligatorios.
Al enviar el formulario, debe llamarse al método correspondiente del servicio.
Si ocurre un error al guardar la reserva, muestra un componente tipo toast con el mensaje de error.

Al ejecutar la app debe salir una vista similar a la siguiente:



Prompt para generar una documentación técnica completa

Genera un Readme.md más completo y detallado que incluya el stack tecnológico empleado en el backend y el frontend, la arquitectectura usada, y toda la información adicional. Subelo despues al Repo de Github

Laboratorio Spring Boot con Cursor
2026
Tecnológico de Antioquia

Ejercicio Propuesto: Fecha de entrega 30 de Octubre de 2026

Siguiendo los pasos realizados, genere una aplicación completa para hacer el traslado de dinero entre cuentas en un banco. Debe mostrar el histórico de transacciones y mostrar además cuando genere el resultado de la transacción un código QR con los datos básicos de la transacción codificados.

Recuerde que puede usar el IDE cursor, windsurf, VS Code o Antigravity IDE para el ejercicio.

Debe entregar informe del ejercicio y código en github y prompts utilizados.